Does Architecture Help or Hinder AI Coding Agents?

A Preregistered Experiment with Layered and Hexagonal Variants of an Evolving Java Service

Tomasz Pisarewski
Fide-Soft
tomasz.pisarewski@fide-soft.com

Draft of October 13, 2026

Abstract. Software architecture exists to help people understand and change code. Coding agents now make a growing share of those changes. Two questions follow. Does a structure designed for human cognition help or hinder an agent? And does the agent keep that structure intact as it works? We report a preregistered controlled experiment (OSF `r5t3d`). We built an HL7 FHIR R4 REST service in Java 21 and Spring Boot 3.5 in three behaviorally equivalent variants: no explicit architecture (None), layered (Layered), and hexagonal, i.e. ports and adapters (Hexagonal). All three pass the same 79-test black-box suite. Two open-weights models, Laguna S 2.1 and Qwen3-Coder-30B-A3B, run in the mini-swe-agent scaffold. Each implements the same 24 product tickets in a fixed order, every ticket on top of its own previous changes, in five independent runs per cell (30 trajectories, 720 attempts). Hidden acceptance tests measure success. Tokens measure cost. After every task we measure erosion, using an architecture-agnostic index and ArchUnit conformance. Success (H3): **[H3-abstract]** (hexagonal minus none: **[H3-hex-none-pp]** percentage points, Holm-adjusted p = **[H3-hex-none-pholm]**). Cost (H1): **[H1-abstract]**. Erosion (H2): **[H2-abstract]**. Conformance (H6): **[H6-abstract]**. Capability interaction (H4): **[H4-abstract]**. The whole experiment ran on a laptop with Docker and cost **[cost-total]** in API fees (budget USD 30). We release all variants, tasks, hidden tests, harness, transcripts, and per-task snapshots.

Keywords: coding agents; software architecture; architecture erosion; hexagonal architecture; controlled experiment; preregistration; open science.

1 Introduction

Software architecture is a tool for human cognition. Modules hide design decisions so that a person can change one part of a system without understanding the rest [25]. Layers and ports separate concerns so that the developer knows where a change belongs and what it may touch [9, 11, 14]. These structures cost something: more files, more indirection, more mapping code. We pay that cost because it buys comprehension and controlled change for human maintainers.

LLM-based coding agents now take on a growing share of maintenance and feature work. They read code through a shell, not through an IDE and a mental model built over months. Two things are unclear. First, does the indirection that helps humans help agents, or does it tax them? Our earlier exploratory study on small Python repositories found a tax: equivalent code with ports and adapters lowered single-task success, more so for weaker models, mainly because the logic to change sat in opaquely named adapter files away from the entry point [27]. That study covered neither realistic project size, nor Java, nor sustained evolution. Second, do agents preserve an architecture while they evolve the code? Architecture erosion, the growing gap between intended and implemented architecture, is a classic problem of long-lived systems [12, 20, 22, 26]. Agents may speed it up. They optimize for local correctness, the ticket at hand and its tests, and may drift from global architectural intent [3]. A recent benchmark has agents extend their own code over checkpoints and measures erosion at each step [6]. None of these studies vary the architecture while holding behavior constant.

We run that experiment. One FHIR R4 service exists in three behaviorally equivalent variants (None, Layered, Hexagonal) that pass the same black-box suite. An agent implements 24 product tickets in order, each on top of its own previous result, and we measure success, cost and erosion after every task. The design, hypotheses, analysis code and sample size were frozen and registered before any confirmatory run. We ask three research questions:

RQ1 *Cost.* Does architecture change how much work an agent needs per task? (H1; by task type, H5)

RQ2 *Success.* Does architecture change whether the agent completes the task, and does that depend on model capability? (H3; H4)

RQ3 *Erosion.* Does an explicit architecture slow structural decay under agent-driven evolution, and do agents keep to the architecture's own rules? (H2; H6)

Contributions.

1. **Controlled, equivalent variants.** Three architectures of one non-trivial service (2.2–4.1 kLOC), with an identical external contract, schema and dependency set, admitted only after a mechanical equivalence gate. Building them in parallel exposed 13 specification ambiguities, which became binding clarifications.
2. **Cumulative evolution with erosion tracking.** Agents work on their own earlier output across 24 tickets. Every task is a git snapshot, graded against all hidden tests so far and measured with a pinned erosion pipeline (ArchUnit, CK, PMD/CPD, SpotBugs).
3. **Preregistration.** Hypotheses, tests, sample size and the analysis script were frozen (git tag `prereg-v1`) and registered on OSF before data collection. This paper was drafted before the results were known. Deviations are logged.
4. **Open, reproducible, cheap.** Open-weights models, an offline Docker sandbox and an unmodified public scaffold (mini-swe-agent 2.4.6). The full experiment cost about USD 0.04 per attempt. All materials, transcripts and snapshots are released.
5. **Lessons for low-budget agent experiments** (§7): failure modes of free and paid endpoints, scaffolds and graders that we hit and fixed before registration.

2 Related Work

Coding-agent benchmarks. SWE-bench [19] established repository-level issue resolution with hidden FAIL_TO_PASS and PASS_TO_PASS tests. We follow that grading scheme. SWE-agent [41] showed that the agent–computer interface matters. mini-swe-agent [34] reduces the scaffold to a bash-only loop and is the reference scaffold of the SWE-bench Bash Only leaderboard [35]. We use it so that scaffold effects stay small and identical across models. Java issue-resolution benchmarks exist [42, 43]. Like most benchmarks they evaluate independent tasks on fixed snapshots of real repositories, so they cannot separate the effect of architecture from everything else that differs between repositories.

Long-horizon evolution and degradation. SWE-EVO [2] poses long-horizon evolution tasks derived from release notes. SlopCodeBench [6] is the closest design to ours: agents extend their own code over checkpoints, and erosion and verbosity are measured at every step. It does not manipulate architecture while holding behavior constant, and that manipulation is what this study adds. The notions of *agentic entropy* [3], where agents optimize local correctness and drift from global architectural intent, and *coherence debt* in repository-scale tasks [5] give conceptual framings for H2 and H6.

Code properties that affect agents. A growing line of work holds the agent fixed and varies the code. Trivedi and Schmitt [37] build minimal pairs that differ in code cleanliness and measure success and token cost with hidden tests. They leave open whether such effects compound over long-horizon evolution, which our cumulative design addresses for architecture. Borg et al. [8] relate file-level CodeHealth to LLM refactoring outcomes. Ustynov [38] revisits conventions written for human readers in light of agentic development and leaves architecture to future work. *Theory of Code Space* [31] asks whether agents can build an architectural map of a code base. Our predecessor study [27] asked whether indirection changes task success and traced the effect mainly to file naming. All of these work with single tasks or single snapshots.

Architecture and agents. Three recent works are closest in topic. *Architecture as Capability Equalizer* [4] varies how an architecture is *specified* to six models, including ArchUnit-style rules, and finds that structured specifications help weaker models most. It does not vary the architecture of the code. A pilot study [1] has three models build a hexagonal microservice and reports that, without automated architectural linting, structural debt accumulates faster with small models. It studies one style with no controlled alternative. In grey literature, Stoyanov [33] compares a flat and a hexagonal version of one Java system on 9 + 6 tasks and reports that the hexagonal version took 37.8% longer to reach acceptance. That comparison involves one system and one author and was not preregistered. Together these works motivate H1, H2 and H6 but do not test them under control.

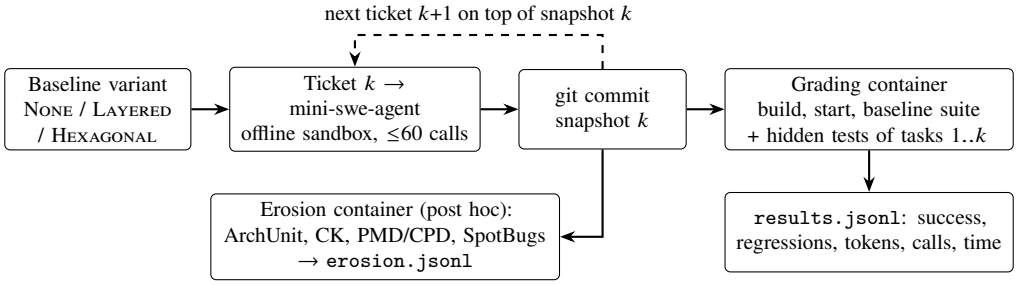


Figure 1: One trajectory. The agent never sees the hidden tests. Grading and erosion measurement run on the committed snapshot in separate offline containers.

Architecture erosion. Perry and Wolf [26] distinguish *erosion*, which violates the architecture, from *drift*, which is insensitivity to it. De Silva and Balasubramaniam [12] survey methods to control erosion, including conformance checking against rules, which we implement with ArchUnit [36]. Li et al. map the erosion literature [22] and report practitioners' views of its causes and symptoms [21]. Common symptoms (dependency cycles, rising coupling, duplication, rule violations) are what our erosion index aggregates. We measure them with established metrics: CK [7, 10], Martin's distance from the main sequence [23], PMD/CPD [28] and SpotBugs [32]. To our knowledge erosion research has so far studied human developers. Agents as the source of erosion are new.

3 Study Design

This section summarizes the frozen protocol (PREREGISTRATION.md at tag `prereg-v1`). Phase 1, reported here, covers the FHIR domain. A second domain (order management) is built with the same method and will be registered separately as Phase 2, where domain generality (H7) is tested. Figure 1 shows the data flow for one trajectory.

3.1 Application and variants

The application is a FHIR R4 REST facade [16] over its own persistence. It covers five resource types (`Patient`, `Practitioner`, `Encounter`, `Observation`, `Appointment`) with read, create, update and a defined subset of search parameters, resource versioning (`meta.versionId`, `ETag`) and `OperationOutcome` errors. Every variant uses Java 21, Spring Boot 3.5.16, Spring Data JPA, H2, Flyway and the HAPI FHIR *structures and parser only* (`hapi-fhir-structures-r4` 8.12.1). HAPI server modules are excluded, because they impose their own architecture and would remove the manipulated variable. Seed data is synthetic (Synthea [39]).

We chose Spring Boot 3.5 rather than 4.x on purpose. Boot 4 (November 2025) changed core APIs (Jackson 3, modularized starters) and is newer than much of the models' training data. It would add a framework-novelty confound unrelated to architecture.

The variants differ only in code organization (ARCHITECTURES.md):

- None: idiomatic "tutorial-style" Spring. Controllers parse, validate, apply rules, map FHIR resources and call Spring Data repositories directly. Packages `controller`, `entity`, `repository`, `util`. Realistic, not deliberately bad.
- Layered: package-by-layer `web` → `service` → `repository` (+ `entity`). HAPI types are confined to `web`.
- Hexagonal: ports and adapters after Hombergs [18] (reference implementation *buckpal*). A pure-Java `domain` with its own clinical model, `application` with inbound/outbound ports and use-case services, and `adapter.in.web`, `adapter.out.persistence` and `config`. FHIR mapping lives only in the web adapter.

The FHIR domain makes the placement of the standard model itself an architectural decision. In None HAPI resource classes run through the whole code base. In Layered they stop at the service boundary. In Hexagonal the domain has its own model and the FHIR mapping sits in one adapter, which is exactly the kind of indirection that was costly in the predecessor study.

Equivalence gate. A variant was admitted only if (i) it passes the shared black-box acceptance suite (79 tests over HTTP against the running jar, with FHIR R4 validation of every response), (ii) its Flyway schema is byte-identical and its dependency set exactly matches the other variants (ArchUnit in test scope is the only extra), (iii) it has zero violations of its own ArchUnit rules, and (iv) its size profile is published (Table 1). The size differences are intrinsic to the styles (ports, mapping, a separate domain model). We report them and do not equalize them. Each variant also contains an `AGENTS.md` with the same build/test commands and a neutral, package-level layout description of equal length (±10%) and no task hints.

Table 1: Size profile of the three baselines at the equivalence gate (2026-10-07). All pass the same 79-test acceptance suite.

Variant	Main LOC	`.java` files	Packages	Own tests (incl. ArchUnit)	`AGENTS.md` words
NONE	2243	27	4	8	173
LAYERED	2677	55	5	14	183
HEXAGONAL	4064	104	12	16	185

Table 2: Task strata. Units = architectural units a change is expected to touch.

Stratum	Tasks	Example	Units
D domain rule	T01 T05 T09 T13 T17 T21	T01: body-weight Observations must use UCUM `kg` or `g`, else 422 `business-rule`	1
F feature	T02 T06 T10 T14 T18 T22	T02: `Observation?date=` search with prefixes; listed in the CapabilityStatement	3+
C cross-cutting	T03 T07 T11 T15 T19 T23	T03: optimistic locking with `If-Match` on every `PUT`; 412 on conflict	many
I integration	T04 T08 T12 T16 T20 T24	T04: POST critical glucose Observations to a configured URL; failure must not fail the create	new adapter

Construction. The specification (`fhir/SPEC.md`) is the single source of truth for behavior. Three AI agents built the three variants in parallel from it, and the author reviewed them. Whenever two builders interpreted the specification differently, the ambiguity became a binding clarification with its own acceptance test, and all variants were re-gated. This produced 13 clarifications (C1–C13), for example the order of checks on `PUT` (parse error before id mismatch before unknown id before business rules), the status for unsupported HTTP methods on known types (405), and the handling of codes that are valid in FHIR R4 but outside the supported subset. The gate caught four real behavioral divergences (C1, C7, C9, C11). We see this as a side result: parallel independent implementation is an effective way to find specification gaps.

3.2 Tasks

There are 24 tasks, written as product tickets. They describe externally observable behavior and never name files, classes, packages or layers, so all three variants receive identical text (1861 words in total). HTTP contracts and configuration property names are part of the external interface and may be named. Every ticket ends with acceptance criteria that the agent can see. The tasks fall into four strata of six, interleaved in a fixed order that mimics product evolution (Table 2).

Hidden tests. Each task has hidden black-box acceptance tests (141 test methods in total, 3–8 per task), the same file for all variants. They run over HTTP against the running application. FAIL_TO_PASS tests fail on every baseline. Guard tests (PASS_TO_PASS) pass on the baseline and stop over-reaching solutions. For integration tasks the grader starts the application with the relevant property pointing to a mock server controlled by the test. `harness/validate_tasks.py` checked on all three baselines that each task's FAIL_TO_PASS tests fail and the baseline suite passes. [TODO: attach the 24-task validation log; data/validate_phase1.log covers 12 tasks only. No reference solutions are in the repository; do not claim them.]

Dependencies and superseded tests. Tasks were designed to be independent: 22 have no dependency, and two have a soft one that their hidden tests do not need (T19 on T03, T20 on T06). Some tickets legitimately change baseline behavior. For example, T18 makes `_count` a valid search parameter, and several tasks extend the CapabilityStatement that a baseline test checks exactly. Nine of the 24 tasks therefore declare which baseline tests they *supersede*. From that task on, those tests are no longer counted as regressions (§3.4).

3.3 Agent and models

Scaffold. mini-swe-agent 2.4.6 [34] with its `DefaultAgent` and a single bash tool through native tool calling. The prompt is identical in every condition and neutral about architecture. It asks the agent to read `AGENTS.md`, find the relevant code, make the change, build and test, and submit. The agent may run the visible tests (`mvn test`) and start the application, and it never sees the hidden tests. The budget is 60 model calls per task. *Success* means solved within that budget: the hidden tests pass on the committed state, whatever the stop reason. Each shell command times out after 600 s. There is no wall-clock limit, because provider latency must never become an agent failure.

Models. Both arms use open-weights models, served through OpenRouter at temperature 0.2 with at most 16,384 output tokens per call:

- *strong*: Poolside Laguna S 2.1 [29] (118B total / 8B active coding mixture-of-experts, OpenMDW-1.1 weights), run *without* extended thinking. With thinking on, the serving provider ignored both the reasoning budget and `effort: low`. Calls ran to the output cap without a tool call, and in a shakedown 6 of 17 attempts ended in repeated format errors. With thinking off there were no format errors in 120 calls. This decision was made on reliability alone, before registration.
- *weak*: Qwen3-Coder-30B-A3B-Instruct [30], the model used in the cloud runs of the predecessor study. A preregistered floor rule would have replaced it with Qwen3-Coder-Next if it had solved fewer than 30% of pilot attempts in NONE. The rule was not triggered.

Open weights matter for reproducibility. Hosted endpoints change or disappear (§7), but anyone can re-run the study with the same weights. The serving provider is logged per call. In the pilot, all strong-arm calls were served by Poolside and all weak-arm calls by SiliconFlow.

3.4 Harness

Sandbox. Each attempt runs in a fresh container of a pre-warmed image (`archagents/sandbox:1`, Temurin JDK 21, Maven with every dependency cached) with `--network none`, 2 CPUs and 3 GB RAM. Maven runs offline. The trajectory's repository is mounted as the workspace. Every command is wrapped in `timeout -k 5 590` inside the container, so a hung build or a forgotten server is killed before the scaffold gives up on the command.

Grading. After the agent stops, the harness commits the workspace. The grader exports the committed tree (`git archive`), builds it in a separate offline container, starts the jar, and runs the baseline acceptance suite plus the hidden tests of the current and all earlier tasks. App configuration accumulates, so earlier integration tasks keep being checked against their mock servers. A task is a *success* if the snapshot builds, the application starts, and every hidden test of the current task passes. A test is a *regression* if it passed at the previous grading and fails now. Tests of earlier tasks that the agent never solved are recorded as *still failing*, not as regressions. Superseded baseline tests are excluded from both.

Infrastructure failures. Provider errors, container crashes and harness exceptions are not agent failures. Only four exit states count as agent outcomes: submitted, step limit reached, time exceeded, and repeated format errors. Any other exit rolls the workspace back to the last commit, and the attempt is retried later, up to three times. After that it is recorded as missing. A trajectory with more than 20% infrastructure failures is re-run from scratch. No data are excluded because of their outcome values.

Scheduling. Task order is fixed. Execution is interleaved: for each task position, the harness visits the trajectories in a seeded random order. Provider drift, throttling or a withdrawn model therefore hits all architecture arms equally. As a logged deviation (§3.7), Phase 1 runs one such interleaving worker per run, two to three in parallel. Each worker contains all six architecture × model trajectories of its run.

3.5 Measures

Success and cost (per attempt). Success (binary), regression flags, stop reason, total tokens (prompt + completion, including cached prompt tokens), model calls, wall time, files changed, and added/deleted lines. Total tokens is the primary cost measure. We also report a recomputed USD cost that prices cached tokens separately.[1]

Erosion (per committed snapshot). `harness/erosion.py` measures every snapshot post hoc in a second pinned image (`archagents/sandbox:2`). The image adds ArchUnit 1.5.1, CK 0.7.0, PMD/CPD 7.28.0 (`quickstart` rule set) and SpotBugs 4.10.4 (default effort, medium threshold), all checked against published SHA-256 digests. Because a snapshot is deterministic, post-hoc measurement gives the same values as measuring inside the loop. CK 0.7.0 cannot parse Java records. The tool therefore parses at JLS 21 and runs CK on a copy in which each record is rewritten as a final class with the same fields. With this change, CK reports no errors on any baseline. The index has seven components (Table 3).

For trajectory t, task index k (0 = baseline) and component m, let $x_{t,k,m}$ be the metric value on the committed snapshot. Then

$$d_{t,k,m} = x_{t,k,m} - x_{t,0,m}, \qquad s_m = \mathrm{SD}_t\left(d_{t,K_t,m}\right), \qquad E_{t,k} = \frac{1}{|M^*|}\sum_{m\in M^*}\frac{d_{t,k,m}}{s_m}, \tag{1}$$

[1]Prices per million tokens at registration (input / output / cache read): strong 0.09 / 0.18 / 0.009, weak 0.07 / 0.28 / 0.07 USD.

Table 3: Components of the erosion index (PREREGISTRATION §4.3).

m	Component	Source
1	package-level dependency cycles in `org.archagents.fhir.*`	ArchUnit slices
2, 3	mean and max coupling between objects (CBO)	CK
4	mean lack of cohesion (LCOM)	CK
5	mean normalized distance from the main sequence $D = \|A + I - 1\|$	ArchUnit metrics
6	static-analysis warnings (PMD violations + SpotBugs bugs)	PMD, SpotBugs
7	duplicated lines (clones ≥100 tokens)	CPD

Table 4: Preregistered hypotheses for Phase 1 and their primary tests. Contrasts: Layered−None, Hexagonal−None, Hexagonal−Layered, Holm-corrected within each hypothesis. H7 (domain generality) needs Phase 2.

	Prediction	Sides	Primary test
H1	cost: None < Layered < Hexagonal (total tokens)	directional	sign-flip test on per-task differences of mean log tokens (24 tasks)
H2	erosion slope lower in Layered and Hexagonal than in None	directional	label permutation of per-trajectory index slopes, stratified by model
H3	success rate differs between architectures	two-sided	sign-flip test on per-task success-rate differences
H4	Hexagonal−None success gap larger for the weak model	directional	sign-flip test on per-task interaction contrasts
H5	Hexagonal cost penalty largest for multi-unit strata (F, C, I), smallest for D	directional	per-stratum H1 contrasts; permutation of the (F,C,I) vs. D contrast
H6	conformance violations accumulate (slope > 0); Layered vs. Hexagonal two-sided	mixed	as H2 on violation-count slopes

where K_t is the last compiling snapshot of t, the standard deviation pools all 30 trajectories across arms and models, and $M^* = \{m : s_m > 0\}$. A component with zero pooled spread is dropped and reported. There is no centering, so $E = 0$ still means "no change since baseline" (centering would add a constant per component and not change any slope). The H2 outcome is the OLS slope β_t of $E_{t,k}$ over k. Snapshots that do not compile have no metric values. They are left out of the slope fits, and their number per arm is reported.

Conformance (H6). For Layered and Hexagonal, the number of violations of the variant's own layer or hexagon rules (ARCHITECTURES.md). They are evaluated by a frozen copy of the baseline rules in the measurement tool, never by the copy in the snapshot, which the agent may have edited. The general checks (field injection, HAPI server modules) and the cycle rule are not counted, since cycles enter the erosion index. A rule whose target package was deleted counts as one violation. The H6 outcome is the OLS slope of this count over k.

3.6 Hypotheses and analysis

Table 4 lists the hypotheses and their tests. All primary tests are permutation tests on the unit of inference [15], implemented in `analysis/analyze.py`, which was frozen with the registration and dry-run on the pilot data. Everything is seeded (20261012).

Task-level tests (H1, H3, H4, H5). The task is the unit of inference. For each task and arm we pool all attempts over runs and models and compute the success rate (H3) or the mean log(total tokens + 1) (H1). For each contrast, the 24 per-task differences enter a two-sided sign-flip permutation test on their mean, with 100,000 random sign patterns including the observed one. When there are 16 or fewer tasks, as in the per-stratum H5 contrasts, all patterns are enumerated. H4 applies the same test to per-task interaction contrasts (weak-model effect minus strong-model effect, in percentage points). H5 permutes stratum labels to test whether the Hexagonal−None cost difference is larger in strata F, C and I than in D. Secondary H1 analyses repeat the test on successful attempts only, model calls, files changed, wall time and recomputed USD.

Trajectory-level tests (H2, H6). We fit one OLS slope per trajectory and compare arms with a two-sided permutation test that shuffles architecture labels within model strata (10 trajectories per arm, 10,000 relabelings).

Table 5: Data collection summary (top) and task success rate by architecture and model, pooled over runs and tasks (bottom).

Runs completed (cost rule)	**[runs-final]**
Trajectories / attempts	**[N-trajectories]** / **[N-attempts]**
Missing after 3 infra retries	**[N-missing]**
Infrastructure retries	**[N-infra-retries]**
Trajectories re-run (>20% infra)	**[N-traj-rerun]**
Submitted / step limit / format err.	**[stop-reasons]**
Non-compiling snapshots N/L/H	**[noncompiling-by-arm]**
Providers (strong / weak)	**[providers]**
Total cost; per attempt	**[cost-total]**; **[cost-per-attempt]**

	None	Layered	Hexagonal
strong	**[succ-none-strong]**	**[succ-lay-strong]**	**[succ-hex-strong]**
weak	**[succ-none-weak]**	**[succ-lay-weak]**	**[succ-hex-weak]**
pooled	**[succ-none]**	**[succ-lay]**	**[succ-hex]**

Inference criteria. $\alpha = 0.05$. The three architecture contrasts within each hypothesis are Holm-corrected [17]. A directional hypothesis counts as supported only if the corrected test is significant *and* the effect has the predicted sign. All effects come with 95% percentile CIs from a cluster bootstrap over tasks with 10,000 resamples [13]. Secondary mixed models estimate adjusted effects: a logistic GLMM `success ~ architecture * model + (1|task) + (1|trajectory)`, LMMs with the same structure for log tokens, and `erosion_index ~ architecture * task_index + (task_index|trajectory)`. A sensitivity analysis adds `dependency_failed` to the success model. Where models and primary tests disagree, both are reported. [TODO: GLMM/LMM and the dependency_failed sensitivity analysis are not implemented in analyze.py; implement exactly as preregistered and label any choices not fixed there.]

Sample size and sensitivity. The design is 3 architectures × 2 models × 5 runs = 30 trajectories of 24 tasks, i.e. 720 attempts. The size was set by resources: the study is self-funded with an API budget of about USD 30. A simulation (`analysis/power_sim.py`; logistic model with task and trajectory random effects calibrated to the pilot) gave the primary H3 test with 24 tasks × 5 runs a power of 0.93 at $\alpha = 0.05$ for a 15 percentage-point difference, and 0.86 at the Holm-adjusted $\alpha = 0.0167$. For 10 points the power is 0.67 and 0.53. Smaller differences are reported as inconclusive, not absent. The simulation also showed that adding tasks raises power more than adding runs, because the task is the unit of inference. This is why we chose 24 tasks × 5 runs over 12 × 10 at equal cost. [TODO: the committed power_sim.py has N_TASKS = 12; archive the 24-task run that produced these numbers.] A preregistered, outcome-blind cost rule would drop run R5 if the forecast spend after R1 exceeded USD 30. Outcome: **[cost-rule-outcome]**.

3.7 Preregistration and open science

Preregistration fixes the confirmatory analysis before the data exist and so separates it from exploration [24]. The open-ended idea was registered on OSF on 2026-10-05 (`https://osf.io/w467e`). The full protocol was registered on 2026-10-13 at 09:25 UTC with the OSF Preregistration template (`https://osf.io/r5t3d`), under embargo until 2027-07-31. The registered materials are git tag `prereg-v1`, commit `3d347cb`. Before registration only harness-debugging data existed: three pilot tasks P1–P3 in all six cells (18 attempts) and one shakedown trajectory over T01–T24 (None, strong model). Both are excluded from every analysis. The models, settings, sandbox images, tasks, hidden tests and analysis script did not change after the freeze. One deviation is logged (DEVIATIONS.md): execution runs in two to three parallel workers, one per run, instead of a single queue over all 30 trajectories, because a sequential queue would take about 70 hours. Every worker interleaves all arms, so the protection against drift is kept. Further deviations: **[deviations-list]**.

4 Results

[TODO: Fill from data/phase1/analysis.json after data collection ends. Report every preregistered test, including non-significant ones.]

4.1 Data collection

Data were collected from **[collection-start]** to **[collection-end]** (Table 5).

4.2 H3: Success

Table 5 gives success rates per cell and Table 6 the primary test. H3 is **[H3-verdict]**. The GLMM odds ratio for Hexagonal vs. None is **[H3-glmm-hex-none]** (profile CI). Adding `dependency_failed` **[H3-depfailed-sensitivity]**. Figure 2a shows success per task.

Table 6: Task-level primary tests (24 tasks): mean per-task difference, 95% cluster bootstrap CI, sign-flip p, Holm-adjusted p. H3 in percentage points of success; H1 in log total tokens (positive = first arm more expensive); H4 is the interaction contrast weak-model effect minus strong-model effect in pp (secondary, no Holm correction; prediction: negative for HEXAGONAL−NONE).

Contrast	Estimate	95% CI	p	p_{Holm}
H3 success (pp), two-sided				
LAYERED−NONE	**[H3-lay-none-pp]**	**[H3-lay-none-ci]**	**[H3-lay-none-p]**	**[H3-lay-none-pholm]**
HEXAGONAL−NONE	**[H3-hex-none-pp]**	**[H3-hex-none-ci]**	**[H3-hex-none-p]**	**[H3-hex-none-pholm]**
HEXAGONAL−LAYERED	**[H3-hex-lay-pp]**	**[H3-hex-lay-ci]**	**[H3-hex-lay-p]**	**[H3-hex-lay-pholm]**
H1 cost (Δ log tokens, all attempts), directional				
LAYERED−NONE	**[H1-lay-none-est]**	**[H1-lay-none-ci]**	**[H1-lay-none-p]**	**[H1-lay-none-pholm]**
HEXAGONAL−NONE	**[H1-hex-none-est]**	**[H1-hex-none-ci]**	**[H1-hex-none-p]**	**[H1-hex-none-pholm]**
HEXAGONAL−LAYERED	**[H1-hex-lay-est]**	**[H1-hex-lay-ci]**	**[H1-hex-lay-p]**	**[H1-hex-lay-pholm]**
H4 capability interaction (pp), directional				
LAYERED−NONE	**[H4-lay-none-pp]**	**[H4-lay-none-ci]**	**[H4-lay-none-p]**	n/a
HEXAGONAL−NONE	**[H4-hex-none-pp]**	**[H4-hex-none-ci]**	**[H4-hex-none-p]**	n/a
HEXAGONAL−LAYERED	**[H4-hex-lay-pp]**	**[H4-hex-lay-ci]**	**[H4-hex-lay-p]**	n/a

Figure pending: `figures/success_by_task.pdf`
(produced from `data/phase1` after unblinding)

(a) success per task and architecture, by model

Figure pending: `figures/cost_by_arm.pdf`
(produced from `data/phase1` after unblinding)

(b) total tokens per attempt (log scale)

Figure 2: Success and cost by architecture. Placeholders until unblinding.

4.3 H1: Cost

H1 is **[H1-verdict]** (Table 6, Figure 2b). The share of attempts that hit the 60-call budget was **[budget-hit-rate-by-arm]** (NONE / LAYERED / HEXAGONAL), which bounds how far cost is censored (§6). The LMM estimate for HEXAGONAL vs. NONE is **[H1-lmm-hex-none]**. Secondary measures are in Table 7.

4.4 H5: Cost by task stratum

H5 is **[H5-verdict]** (Table 7).

4.5 H4: Capability interaction

H4 is **[H4-verdict]** (Table 6).

4.6 H2: Erosion

The index used the components **[H2-metrics-used]**. Components dropped for zero pooled spread: **[H2-metrics-dropped]**. Table 8 compares slopes, and Figure 3a shows the index over task position. H2 is **[H2-verdict]**. Per-component slopes are given in the supplementary material (**[H2-component-table]**). LMM architecture × task-index interaction: **[H2-lmm-interaction]**.

4.7 H6: Conformance

Own-architecture violations changed by **[H6-lay-slope]** per task in LAYERED and **[H6-hex-slope]** in HEXAGONAL (difference **[H6-hex-lay-diff]**, p = **[H6-hex-lay-p]**; Figure 3b). H6 is **[H6-verdict]**. The rules violated most often were **[H6-top-rules]**. [TODO: analyze.py tests only the HEX-vs-LAY slope difference for H6; add the preregistered one-sample test of slope > 0 per arm, labeled as such.]

4.8 Exploratory analyses

These analyses were preregistered as exploratory. *Regressions:* **[regression-rate-by-arm]** of graded attempts broke at least one previously passing test. *Localization:* **[localization-summary]** (files read before the first edit vs. files where the change landed). *Qualitative coding* of a stratified sample of failed attempts: **[qualitative-summary]**.

Table 7: H1 secondary cost measures and H5 (cost by stratum). Cells: mean per-task difference [95% CI], p. H5 strata have six tasks each (exact sign-flip test; smallest attainable two-sided $p = 0.031$).

H1 secondary measures (Holm-adjusted p)			
Measure	LAYERED−NONE	HEXAGONAL−NONE	HEXAGONAL−LAYERED
log tokens, successful attempts	**[H1sec-succtok-lay-none]**	**[H1sec-succtok-hex-none]**	**[H1sec-succtok-hex-lay]**
model calls	**[H1sec-calls-lay-none]**	**[H1sec-calls-hex-none]**	**[H1sec-calls-hex-lay]**
files modified	**[H1sec-files-lay-none]**	**[H1sec-files-hex-none]**	**[H1sec-files-hex-lay]**
wall time (s)	**[H1sec-wall-lay-none]**	**[H1sec-wall-hex-none]**	**[H1sec-wall-hex-lay]**
recomputed USD	**[H1sec-usd-lay-none]**	**[H1sec-usd-hex-none]**	**[H1sec-usd-hex-lay]**
H5: HEXAGONAL−NONE Δ log tokens by stratum			
Stratum	Estimate	95% CI	p
D domain rule	**[H5-D-est]**	**[H5-D-ci]**	**[H5-D-p]**
F feature	**[H5-F-est]**	**[H5-F-ci]**	**[H5-F-p]**
C cross-cutting	**[H5-C-est]**	**[H5-C-ci]**	**[H5-C-p]**
I integration	**[H5-I-est]**	**[H5-I-ci]**	**[H5-I-p]**
(F, C, I) − D	**[H5-multi-minus-D]**		**[H5-multi-minus-D-p]**

Table 8: Trajectory-level tests: mean per-trajectory OLS slope per task for each arm of the contrast, and label-permutation tests stratified by model (10 vs. 10 trajectories). H2: erosion index. H6: own-architecture conformance violations.

Contrast	slope (first arm)	slope (second arm)	p	p_{Holm}
H2 erosion index, directional				
LAYERED−NONE	**[H2-lay-none-a]**	**[H2-lay-none-b]**	**[H2-lay-none-p]**	**[H2-lay-none-pholm]**
HEXAGONAL−NONE	**[H2-hex-none-a]**	**[H2-hex-none-b]**	**[H2-hex-none-p]**	**[H2-hex-none-pholm]**
HEXAGONAL−LAYERED	**[H2-hex-lay-a]**	**[H2-hex-lay-b]**	**[H2-hex-lay-p]**	**[H2-hex-lay-pholm]**
H6 conformance violations, two-sided				
HEXAGONAL−LAYERED	**[H6-hex-slope]**	**[H6-lay-slope]**	**[H6-hex-lay-p]**	n/a

5 Discussion

[TODO: Write after unblinding. Keep to what the preregistered tests support. The skeleton below lists what each outcome pattern would need to address. No claims are made here.]

If architecture protects (lower erosion slope in LAYERED/HEXAGONAL, success not lower).

- Which erosion components carry the effect: cycles and coupling, or warnings and duplication? The baselines differ in size and structure, so a difference in components that scale with code size needs care.
- Is the protection bought with cost (H1)? Report the price per task and per point of erosion avoided.
- Did conformance hold (H6 slope near zero), or did agents keep the metrics flat while breaking layer rules?
- Relation to [1, 4]: does the code's structure act as an implicit specification?

If the agent breaks the architecture (H6 slope > 0, H2 not supported).

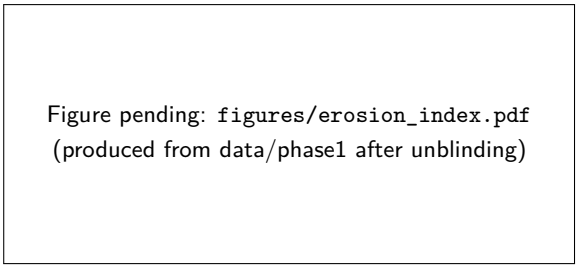
- Which rules break first (e.g. HAPI types leaking into `domain/service`, adapters calling services directly)? Link to the qualitative coding.
- Is the structural cost of HEXAGONAL still paid (H1) while its benefits erode? That is the worst case for practice.
- Interaction with capability (H4) and stratum (H5): do cross-cutting and integration tasks drive the violations?
- Implications for guardrails (enforced ArchUnit rules, instructions). This motivates Study 2.

If results are mixed or inconclusive.

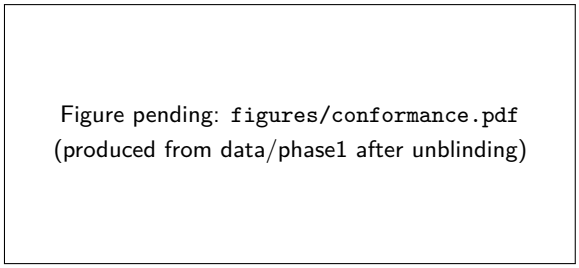
- Use the sensitivity analysis: effects below about 10–15 pp are outside what this design can detect.
- Compare with the predecessor study's single-task tax, and say whether cumulative, larger-scale work changes the picture.

6 Threats to Validity

We follow the usual classification [40]. The first items were stated in the preregistration (§7.1). The rest were identified while running the study.



(a) erosion index $E_{t,k}$, mean and 95% band



(b) own-architecture violations (LAYERED, HEXAGONAL)

Figure 3: Erosion and conformance over task position, per architecture and model. Placeholders until unblinding.

Construct validity. *Size is part of the treatment.* HEXAGONAL has 1.81× the LOC and 3.9× the files of NONE. We report the profile and do not equalize it, so effects of architecture cannot be separated from the code volume the style implies. *Erosion index.* The index aggregates established metrics with equal weight. Some components, such as distance from the main sequence, depend on package structure and may behave differently by style. We therefore report every component separately. *Conformance* exists only for LAYERED and HEXAGONAL. *Cost* includes cached prompt tokens, which are cheap in money but still measure how much context the agent processes. We report recomputed USD as a secondary measure. *Step-limit censoring.* Cost is right-censored at 60 calls. In the shakedown the strong model used the full budget in 18 of 24 tasks. Censoring lowers the power of H1 and compresses differences between arms. It does not bias the permutation test under the null. The secondary analysis on successful attempts is only partly a remedy, because success is itself an outcome. *Excluded snapshots.* Non-compiling snapshots are left out of the slope fits. If arms differ in compile rates, slopes describe different subsets. We report the counts per arm.

Internal validity. *Construction bias.* One author, assisted by AI agents, built all variants and tasks, and no independent human review took place. Mitigations: a public reference style, the mechanical equivalence gate (79 black-box tests, identical schema and dependencies), frozen ArchUnit rules, published size profiles and full release of all materials. AI-built variants may also be closer to what models find natural than human-written code would be. *Task authoring.* Tickets are behavior-only and graded by identical black-box tests, but nobody else reviewed them. *Provider drift.* Hosted models can change silently, and providers can re-route requests. We pinned model IDs, interleaved all arms in every worker, and logged the provider per call. Drift during the collection window affects all arms equally but could still interact with architecture. *Dependencies between tasks.* A failed task can make later tasks harder in that trajectory. We report the preregistered `dependency_failed` sensitivity analysis.

External validity. *Single domain in Phase 1.* Results come from one application. FHIR is a public standard that models have probably seen often, and its external model makes the placement of mapping code especially salient. Phase 2 (order management) tests generality (H7). We claim nothing about other languages, frameworks or much larger systems. *Model choice.* Two open-weights models in one scaffold, chosen partly for cost, with the strong model run without extended thinking. Frontier proprietary agents with richer tools may behave differently, and with two models H4 is a contrast between two points, not a capability curve. *Scaffold.* A bash-only loop with a 60-call budget is deliberately minimal. Products with planning, retrieval or self-review steps may close or widen any gap.

Conclusion validity. 24 tasks give adequate power only for differences of about 15 pp or more (§3.6). H5 per-stratum tests have six tasks each and are descriptive. Contamination is limited by keeping all materials private until data collection ends and by running without network access. The materials will become public afterwards.

7 Lessons for Running Agent Experiments on a Shoestring

The study ran on one laptop with Docker and a budget of USD 30. Most of the engineering effort went into problems that a well-funded lab may never see but that can silently bias a low-budget experiment. We record them because each one would have turned infrastructure behavior into apparent agent behavior.

1. Free endpoints are not infrastructure. We first planned to use free (`:free`) endpoints. `qwen/qwen3-coder:free` was withdrawn on 2026-10-11, and `poolside/laguna-s-2.1:free` was throttled for many minutes per call. Both were dropped before any confirmatory data in favor of the paid endpoints of the same models. With provider-side prompt caching the paid runs were cheap: 83% of prompt tokens in the pilot and 96–97% in the

shakedown were served from cache, and Phase 1 cost about USD 0.04 per attempt. Open weights mean the study stays reproducible when endpoints change.

2. Providers may ignore reasoning settings. With extended thinking enabled, the provider ignored both `reasoning.max_tokens` and `effort: low`. Calls ran to the output cap without a tool call, and the scaffold counted them as format errors. Six of 17 shakedown attempts ended this way, a failure mode unrelated to the code base that would have added noise to every arm. Turning thinking off removed it (0 format errors in 120 calls). Measure the format-error rate in a shakedown, and fix reasoning settings before registration on reliability grounds alone.

3. Errors can arrive with HTTP 200. Some upstream errors came back as HTTP 200 with an `error` body and no `choices`. The stock scaffold crashed on them with a `KeyError`. A thin wrapper turns them into retryable API errors and raises the request timeout from 60 to 300 s. More generally, the harness keeps an explicit allow-list of exit states that count as agent outcomes. Everything else is infrastructure: it is rolled back to the last commit and retried, never scored.

4. Command timeouts do not kill processes. When a command times out, the scaffold stops waiting, but a hung `mvn test` or a server the agent started in the background can keep running inside the container. It then competes for the 2-CPU budget and inflates the wall time of later commands. Wrapping every command in `timeout -k 5 590` inside the container, 10 s before the scaffold's own 600 s limit, and removing the container after each attempt fixed this.

5. Define "regression" with care. In a cumulative design, the naive definition (any earlier hidden test that fails now) counts every earlier task the agent never solved as a regression in every later attempt. We define a regression as a test that passed at the previous grading and fails now, record "still failing" tests separately, and compare against the last snapshot that built and started.

6. Cumulative grading needs *supersedes*. Legitimate tickets change baseline behavior: a new search parameter that used to be rejected, a stricter rule that an old test violates, an extended CapabilityStatement that an old test checks exactly. Without an explicit list of superseded tests, correct solutions would register as regressions. Nine of 24 tasks needed such entries. A full shakedown trajectory over all 24 tasks, run before registration, found these conflicts. Configuration must also accumulate, so that earlier integration tasks keep being tested against their mock servers.

7. Make every step resumable and idempotent. Long runs on a laptop get interrupted. If the process is killed between writing a result row and saving trajectory state, a naive restart grades the same task twice. The runner reconciles state with the results file at startup. Measuring erosion post hoc from git snapshots, rather than in the loop, means a crashed or upgraded measurement tool never costs agent runs.

8. Pin and adapt the measurement tools. CK 0.7.0 predates Java records. It either mis-parsed Java 21 syntax or lost every type in a file that contains a record (22 files in the HEXAGONAL baseline, 3–4 in the others). Without the desugaring step (§3.5), coupling and cohesion would have been computed on an incomplete set of types that differs by variant. We pinned tool versions in a separate image tag, checked release digests, and verified that CK reports no errors on the baselines.

8 Conclusion

We set out to test, under preregistered control, whether layered and hexagonal architecture help or hinder AI coding agents that evolve a Java service over many tasks, and whether the agents preserve the architecture. **[conclusion-summary]**. [TODO: Two to three sentences after unblinding, limited to supported hypotheses; state Phase 2 and Study 2 (guardrails) as next steps.]

Data availability. All materials are released under MIT (code) and CC-BY 4.0 (text, data): the three variants, specification and clarifications, tasks, hidden tests, harness, agent and model configuration, sandbox Dockerfiles, raw agent transcripts, per-task git snapshots, results and erosion tables, and analysis scripts. All data are synthetic. OSF project: `https://osf.io/rneym`. Registrations: `https://osf.io/r5t3d` (Phase 1, embargoed until 2027-07-31) and `https://osf.io/w467e` (idea). Code: [TODO: GitHub URL and Zenodo DOI].

Use of AI tools. AI coding agents built the three variants from the author's specification under the author's review (§3.1). [TODO: Disclose AI assistance in harness development and in drafting this manuscript, following the target venue's policy.]

References

[1] Quantitative analysis of technical debt and pattern …. arXiv:2512.04273, 2025. Title truncated in our notes; authors not recorded. TODO verify.

[2] SWE-EVO: long-horizon software evolution from release notes. arXiv:2512.18470. https://github.com/SWE-EVO/SWE-EVO, 2025. Title paraphrased from the repository; authors unknown. TODO verify.

[3] Beyond the 'diff': Addressing agentic entropy. arXiv:2604.16323, 2026. Authors not recorded. TODO verify.

[4] Architecture as capability equalizer for coding agents. arXiv:2608.21747, 2026. Authors not recorded. TODO verify.

[5] The working set of a coding agent: Coherence debt in repository-scale tasks. arXiv:2608.16630, 2026. Authors not recorded. TODO verify.

[6] SlopCodeBench. arXiv:2603.24755. https://github.com/SprocketLab/slop-code-bench, 2026. Full title and authors not recorded. TODO verify.

[7] Maurício Aniche. Java code metrics calculator (CK). https://github.com/mauricioaniche/ck, 2015. Version 0.7.0 used.

[8] Markus Borg et al. Code for machines, not just humans: Quantifying AI-friendliness with code health metrics. In *Proceedings of the ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*, 2026. arXiv:2601.02200. TODO verify author list and venue.

[9] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Wiley, 1996.

[10] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[11] Alistair Cockburn. Hexagonal architecture. https://alistair.cockburn.us/hexagonal-architecture/, 2005.

[12] Lakshitha de Silva and Dharini Balasubramaniam. Controlling software architecture erosion: A survey. *Journal of Systems and Software*, 85(1):132–151, 2012.

[13] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[14] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[15] Phillip Good. *Permutation, Parametric and Bootstrap Tests of Hypotheses*. Springer, 3rd edition, 2005.

[16] HL7 International. FHIR release 4 (v4.0.1). https://hl7.org/fhir/R4/, 2019.

[17] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[18] Tom Hombergs. *Get Your Hands Dirty on Clean Architecture*. Packt, 2nd edition, 2023. Reference implementation: https://github.com/thombergs/buckpal. TODO verify edition used.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[20] Meir M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980.

[21] Ruiyin Li, Mohamed Soliman, Peng Liang, and Paris Avgeriou. Understanding architecture erosion: The practitioners' perceptive. In *Proceedings of the IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pages 311–322, 2021. TODO verify author order and pages.

[22] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process*, 34(3):e2423, 2022.

[23] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2002.

[24] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

[25] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[26] Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992.

[27] Tomasz Pisarewski. The indirection tax: Architectural decoupling, file naming, and the semantic localization of code by coding agents. Unpublished manuscript; materials to be released with this study, 2026.

[28] PMD contributors. PMD: An extensible cross-language static code analyzer. https://pmd.github.io, 2026. Version 7.28.0 used, including CPD.

[29] Poolside. Laguna S 2.1 model card. OpenRouter model `poolside/laguna-s-2.1`, 2026. 118B total / 8B active parameters, OpenMDW-1.1 weights. TODO verify source and URL.

[30] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.

[31] G. Sapunov. Theory of code space: Do code agents understand software architecture? arXiv:2603.00601, 2026. TODO verify.

[32] SpotBugs contributors. SpotBugs: Find bugs in Java programs. https://spotbugs.github.io, 2026. Version 4.10.4 used.

[33] K. Stoyanov. I expected hexagonal architecture to help my coding agent. It didn't. Blog post, https://dev.to/kstoyanovai/i-expected-hexagonal-architecture-to-help-my-coding-agent-it-didnt-3blg, with a technical report on Zenodo, 2026. Grey literature. Full name and Zenodo DOI missing. TODO verify.

[34] SWE-agent team. mini-swe-agent: The 100-line AI agent that solves GitHub issues. https://github.com/SWE-agent/mini-swe-agent, 2025. Version 2.4.6 used. TODO verify repository title and year.

[35] SWE-bench team. SWE-bench bash only leaderboard. https://www.swebench.com, 2025. Accessed 2026-10. TODO verify.

[36] TNG Technology Consulting. ArchUnit: Unit test your Java architecture. https://www.archunit.org, 2025. Version 1.5.1 used.

[37] Trivedi and Schmitt. Does code cleanliness affect coding agents? A controlled minimal-pair study. arXiv:2605.20049, 2026. SonarSource. First names missing. TODO verify.

[38] Ustynov. Beyond human-readable: Rethinking software engineering conventions for the agentic development era. arXiv:2604.07502, 2026. First name missing. TODO verify.

[39] Jason Walonoski, Mark Kramer, Joseph Nichols, Andre Quina, Chris Moesel, Dylan Hall, Carlton Duffett, Kudakwashe Dube, Thomas Gallagher, and Scott McLachlan. Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association*, 25(3):230–238, 2018.

[40] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[42] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. SWE-bench-java: A GitHub issue resolving benchmark for Java. arXiv:2408.14354, 2024. TODO verify author list.

[43] Daoguang Zan et al. Multi-SWE-bench: A multilingual benchmark for issue resolving. arXiv:2504.02605, 2025. TODO verify.